

CSCI-UA 102 — Quiz 2

Stacks

Name: _____ NetID: _____

Asteroid Collision

An array of integers represents asteroids in a row. For each asteroid, the **absolute value** is its size and the **sign** is its direction: positive means moving **right**, negative means moving **left**. All asteroids move at the same speed.

When two asteroids moving in **opposite directions** meet, the smaller one explodes. If they are the same size, both explode. Two asteroids moving in the same direction never meet.

Write a method `int[] asteroidCollision(int[] asteroids)` that returns the state of the asteroids after all collisions, in their original order.

Examples:

- `[5, 10, -5] → [5, 10]` (10 and `-5` collide; `-5` explodes)
- `[8, -8] → []` (same size, both explode)
- `[-2, -1, 1, 2] → [-2, -1, 1, 2]` (no collisions occur)
- `[10, 2, -5] → [10]` (`-5` destroys 2, then loses to 10)
- `[15, 3, 2, 1, -5] → [15]` (`-5` destroys 1, 2, and 3, then loses to 15)

You may use the following Stack interface and assume a working implementation exists:

```
public interface Stack<E> {  
    E pop();           // remove and return the top element  
    void push(E e);    // add element to the top  
    int size();        // number of elements in the stack  
}
```

What is the time complexity of your solution? _____

Solution

```
public static int[] asteroidCollision(int[] asteroids) {
    Stack<Integer> s = new LinkedStack<>();
    for (int a : asteroids) {
        boolean alive = true;
        while (alive && a < 0 && s.size() > 0) {
            int top = s.pop();
            if (top < 0) {
                // both going left -- no collision
                s.push(top);
                break;
            }
            // top > 0, a < 0: collision
            if (top > -a) {
                s.push(top);        // top survives
                alive = false;
            } else if (top == -a) {
                alive = false;      // both die
            }
            // else: top < -a, top dies, loop continues
        }
        if (alive) s.push(a);
    }
    int[] result = new int[s.size()];
    for (int i = result.length - 1; i >= 0; i--)
        result[i] = s.pop();
    return result;
}
```

Time complexity: $O(n)$ — each asteroid is pushed and popped at most once.